QUADRO

Uma linguagem para desenhar — especificação e Fase 1

Disciplina: Teoria da Computação e Compiladores
Produto: analisador léxico, sintático e geração de árvore sintática abstrata (AST)
Implementação: JavaScript com módulos nativos, sem bibliotecas de análise

> Esta entrega cobre exclusivamente a Fase 1: análise de texto e construção de AST. Não desenha figuras nem gera SVG. A estrutura parte do tutorial do Robô e aplica a transformação do parser para uma AST, conforme os materiais da aula.

1. Descrição da linguagem

Quadro é uma linguagem textual mínima para representar uma tela e uma sequência de elementos geométricos. Todo programa deve começar com `quadro L A`, onde `L` é a largura e `A` é a altura. Em seguida, aparece pelo menos um elemento, que pode ser uma reta, um círculo, um retângulo ou um bloco. O bloco permite agrupar elementos e outros blocos em profundidade arbitrária.

Comandos e argumentos têm palavras-chave em minúsculas, sem acentos, e são separados por espaços ou quebras de linha. Chaves `{` e `}` delimitam blocos; o caractere `#` é reservado às cores hexadecimais. Comentários começam com `//` e terminam na quebra de linha. Não existem ponto e vírgula, variáveis, laços, caminhos nem expressões.

A sintaxe **difere da demonstração ilustrativa do enunciado**: a declaração é `quadro`, os grupos são `bloco { ... }`, a forma linear é `reta` e os modificadores são `girar` e `mover`.

1.1 Comandos

Construção	Forma textual	Significado
Tela	`quadro largura altura`	Declara as dimensões obrigatórias do desenho.
Reta	`reta x1 y1 x2 y2`	Segmento entre dois pontos.
Círculo	`circulo cx cy raio`	Centro e raio.
Retângulo	`retangulo x y largura altura`	Origem, largura e altura.
Bloco	`bloco [modificadores] { elementos }`	Agrupa formas e outros blocos.
Estilo	`traco cor`, `preencher cor`, `espessura n`	Atributos opcionais aplicáveis a formas.
Modificador	`girar angulo`, `mover dx dy`	Alterações opcionais aplicáveis apenas a blocos.

Os modificadores `girar` e `mover` podem aparecer em qualquer ordem, mas no máximo uma vez cada. Sua ordem deve ser preservada na AST. Estilos podem aparecer em qualquer ordem; caso um atributo se repita, prevalece a última ocorrência daquele atributo.

1.2 Programa de exemplo A — casa com detalhes

```
quadro 480 320
// Casa e sol
retangulo 90 130 210 150 preencher amarelo traco preto espessura 3
reta 90 130 195 50 traco vermelho espessura 4
reta 195 50 300 130 traco vermelho espessura 4
circulo 385 60 32 preencher #FFD700
bloco mover 118 195 {
  retangulo 0 0 45 85 preencher roxo
  bloco girar 30 {
    reta 10 15 30 15 traco branco espessura 2
  }
}
```

A declaração define uma tela de 480 por 320. O programa contém quatro formas no topo e um bloco deslocado. Dentro dele há um retângulo e outro bloco, que contém uma reta. O primeiro bloco demonstra o deslocamento; o bloco interno demonstra o aninhamento com rotação.

1.3 Programa de exemplo B — ordem dos modificadores

```
quadro 200 160
// A ordem dos modificadores importa.
bloco girar 90 mover 20 10 {
  circulo 0 0 15 preencher azul
  reta 0 0 40 0 traco #FF8800 espessura 2
}
bloco mover 10 20 girar 45 {
  retangulo 0 0 50 25 traco verde
}
```

Os dois blocos usam os mesmos tipos de modificadores, mas em ordens distintas. Isso é registrado como vetores ordenados na AST, sem interpretar as transformações nesta fase.

2. Definição léxica

O analisador lê a entrada da esquerda para a direita. Cada expressão regular usa a **flag** `y` (sticky), de modo que só reconhece um token na posição atual. Palavras-chave, cores nomeadas e números utilizam a condição de término `(?=$|[\s{}]|\/\/)`, para impedir reconhecimento de prefixos de palavras maiores. Assim, `retanguloExtra` não é dividido em `retangulo` + `Extra`.

Token	Expressão regular JavaScript	Descrição / tratamento
`QUADRO`	`/quadro(?=$|[\s{}]|\/\/)/y`	Palavra da declaração obrigatória.
`RETA`	`/reta(?=$|[\s{}]|\/\/)/y`	Palavra-chave de reta.
`CIRCULO`	`/circulo(?=$|[\s{}]|\/\/)/y`	Palavra-chave de círculo.
`RETANGULO`	`/retangulo(?=$|[\s{}]|\/\/)/y`	Palavra-chave de retângulo.
`BLOCO`	`/bloco(?=$|[\s{}]|\/\/)/y`	Palavra-chave de agrupamento.
`GIRAR`	`/girar(?=$|[\s{}]|\/\/)/y`	Modificador de rotação.
`MOVER`	`/mover(?=$|[\s{}]|\/\/)/y`	Modificador de deslocamento.
`TRACO`	`/traco(?=$|[\s{}]|\/\/)/y`	Cor do contorno.

Token	Expressão regular JavaScript	Descrição / tratamento
`PREENCHER`	`/preencher(?=$|[\s{}]|\/\/)/y`	Cor do preenchimento.
`ESPESSURA`	`/espessura(?=$|[\s{}]|\/\/)/y`	Espessura do contorno.
`COR_NOME`	`/(?:vermelho|verde|azul|preto|branco|amarelo|roxo)(?=$|[\s{}]|\/\/)/y`	Sete cores nomeadas aceitas.
`COR_HEX`	`/#[A-Fa-f0-9]{6}(?=$|[\s{}]|\/\/)/y`	Cor hexadecimal exatamente `#RRGGBB`.
`NUM`	`/(?:0|[1-9][0-9]*)(?=$|[\s{}]|\/\/)/y`	Inteiro sem sinal, incluindo `0`; proíbe zeros à esquerda.
`ABRE`	`/\{/y`	Início de bloco.
`FECHA`	`/\}/y`	Fim de bloco.
`COMENTARIO`	`/\/\/[^\r\n]*/y`	**Ignorado**; do `//` até o final da linha.
`ESPACO`	`/[ \t\r\n]+/y`	**Ignorado**; espaços, tabs e quebras de linha.

Os tokens produzidos para a página têm a estrutura `{ tipo, lexema, linha }`. A contagem inicia na linha 1 e considera `\n`, `\r\n` ou `\r` como uma quebra. Regras ignoradas não entram no vetor de tokens, mas suas quebras de linha continuam contando.

Se não houver regra aplicável na posição atual, o lexer registra `{ linha, trecho }`, onde `trecho` inclui toda a sequência inválida até o próximo espaço em branco, e prossegue a varredura. Na presença de **qualquer erro léxico**, o parser não é executado.

3. Definição sintática — gramática livre de contexto

A gramática é `G = (V, Σ, P, S)`, com símbolo inicial **`S = programa`**. Os terminais `Σ` são `QUADRO`, `RETA`, `CIRCULO`, `RETANGULO`, `BLOCO`, `GIRAR`, `MOVER`, `TRACO`, `PREENCHER`, `ESPESSURA`, `COR_NOME`, `COR_HEX`, `NUM`, `ABRE`, `FECHA` e o marcador final lógico `FIM`. `FIM` é verificado pelo parser quando não restam tokens; ele não é gerado pelo lexer. Os não terminais `V` são todos os símbolos minúsculos à esquerda das produções abaixo.

```
programa       -> QUADRO NUM NUM elementos FIM

elementos      -> elemento cauda
cauda          -> elemento cauda | ε

elemento       -> formaReta | formaCirculo | formaRetangulo | grupo
formaReta      -> RETA NUM NUM NUM NUM estilos
formaCirculo   -> CIRCULO NUM NUM NUM estilos
formaRetangulo -> RETANGULO NUM NUM NUM NUM estilos

grupo          -> BLOCO modificadores ABRE elementos FECHA

modificadores  -> GIRAR NUM aposGirar
                | MOVER NUM NUM aposMover
                | ε
aposGirar      -> MOVER NUM NUM | ε
aposMover      -> GIRAR NUM | ε

estilos        -> estilo estilos | ε
```

```
estilo          -> TRACO cor | PREENCHER cor | ESPESSURA NUM
cor             -> COR_NOME | COR_HEX
```

A gramática **não tem recursão à esquerda**. O tipo de `elemento`, `estilo`, `cor` e a primeira alternativa de `modificadores` pode ser escolhido observando apenas o próximo token; portanto, serve à implementação descendente recursiva solicitada. Em `elementos`, o `FECHA` de um bloco ou o fim do programa encerram a lista, mas a primeira ocorrência de `elemento` é obrigatória. Os não terminais `aposGirar` e `aposMover` impedem repetições de modificadores e mantêm a escolha por um token de antecipação.

4. Estrutura da AST

A AST contém valores semânticos, e não tokens. O parser devolve uma raiz `programa`, um objeto com as dimensões da tela e um vetor `comandos` que mantém a ordem do texto. Cada forma inclui um objeto `estilo`, possivelmente vazio, que contém somente os atributos presentes. Cada bloco inclui um vetor `modificadores` na ordem original e um vetor `corpo` com seus elementos. Números são convertidos com `Number(...)`.

Tipo de nó	Campos	Significado
`programa`	`tipo`, `tela`, `comandos`	Raiz e lista ordenada de elementos.
`tela`	`tipo`, `largura`, `altura`	Dimensões declaradas.
`reta`	`tipo`, `x1`, `y1`, `x2`, `y2`, `estilo`	Dois pontos e atributos opcionais.
`circulo`	`tipo`, `cx`, `cy`, `raio`, `estilo`	Centro, raio e atributos opcionais.
`retangulo`	`tipo`, `x`, `y`, `largura`, `altura`, `estilo`	Origem, dimensões e atributos opcionais.
`grupo`	`tipo`, `modificadores`, `corpo`	Modificadores ordenados e elementos internos.
`rotacao`	`tipo`, `angulo`	Modificador com ângulo.
`deslocamento`	`tipo`, `dx`, `dy`	Modificador com deslocamento horizontal e vertical.
(objeto de estilo)	`traco?`, `preenchimento?`, `espessura?`	Somente os atributos fornecidos na forma.

As chaves `{` e `}`, palavras-chave da linguagem e objetos de token não aparecem na AST. Não há AST parcial: se houver erro léxico ou sintático, só é apresentado o erro. A execução e o desenho estão fora desta fase.

4.1 AST completa do programa de exemplo B

A saída abaixo também está no arquivo `AST_exemplo_2.json`:


```
{
  "tipo": "programa",
  "tela": {
    "tipo": "tela",
    "largura": 200,
    "altura": 160
  },
  "comandos": [
    {
```

```
      "tipo": "grupo",
      "modificadores": [
        {
          "tipo": "rotacao",
          "angulo": 90
        },
        {
          "tipo": "deslocamento",
          "dx": 20,
          "dy": 10
        }
      ],
      "corpo": [
        {
          "tipo": "circulo",
          "cx": 0,
          "cy": 0,
          "raio": 15,
          "estilo": {
            "preenchimento": "azul"
          }
        },
        {
          "tipo": "reta",
          "x1": 0,
          "y1": 0,
          "x2": 40,
          "y2": 0,
          "estilo": {
            "traco": "#FF8800",
            "espessura": 2
          }
        }
      ]
    },
    {
      "tipo": "grupo",
      "modificadores": [
        {
          "tipo": "deslocamento",
          "dx": 10,
          "dy": 20
        },
        {
          "tipo": "rotacao",
          "angulo": 45
        }
      ],
      "corpo": [
        {
          "tipo": "retangulo",
          "x": 0,
          "y": 0,
          "largura": 50,
          "altura": 25,
          "estilo": {
            "traco": "verde"
          }
        }
      ]
    }
  ]
}
```

5. Decisões de projeto

Questão aberta	Decisão tomada e consequência
Sintaxe	`quadro`, `reta`, `circulo`, `retangulo`, `bloco`; as chaves delimitam grupos. Não se usam `;` nem `,`.
Declaração de tela	É obrigatória, única, no início, com dois números; depois dela deve haver ao menos uma forma ou grupo.
Números	Aceita `0` e inteiros positivos sem sinal, sem zeros à esquerda (`05` é erro léxico). Valores precisam caber como inteiros seguros em JavaScript (`Number.isSafeInteger`).
Dimensões e coordenadas zero	Aceitas na Fase 1, inclusive tela ou raio zero, porque validações de significado não fazem parte da tarefa.
Cores	Nomes aceitos: `vermelho`, `verde`, `azul`, `preto`, `branco`, `amarelo` e `roxo`. Hexadecimal aceita seis dígitos e é normalizado para letras maiúsculas na AST.
Comentários	`//` até a quebra de linha; nunca entra nos tokens. O `#` fica reservado à cor hexadecimal.
Grupos	Um bloco exige pelo menos um elemento (grupo vazio é erro sintático); suporta aninhamentos sucessivos.
Modificadores	`girar` exige um número; `mover` exige dois; só podem aparecer depois de `bloco` e antes de `{`. Cada um aparece no máximo uma vez; as duas ordens são aceitas e preservadas em um vetor.
Estilos de formas	`traco`, `preencher` e `espessura` aparecem apenas depois dos argumentos de formas; cada um é opcional e pode ser reordenado.
Atributos repetidos	São aceitos; o último valor atribuído ao mesmo atributo prevalece na AST. Ex.: `preencher azul preencher verde` resulta em `preenchimento: "verde"`.
Palavras-chave	Sensíveis a maiúsculas/minúsculas; escritos em minúsculas e sem acentos. O lexer verifica limites de palavra, evitando reconhecimento parcial.
Espaçamento	Espaços, tabulações e quebras de linha funcionam como separadores; chaves podem estar coladas a tokens válidos.
Erros	Erros léxicos indicam linha e palavra inteira inválida; erros sintáticos interrompem o parser no primeiro erro, indicando linha, esperado e encontrado.
AST	Sem pontuação, sem tokens e com valores numéricos; aninhamento e ordem dos modificadores são mantidos.

6. Arquitetura da implementação

1. **`tokens.js`** define expressões regulares com a flag `y`, inclusive regras ignoradas.
2. **`lexer.js`** caminha pelo texto, registra `tipo`, `lexema` e `linha`, e acumula erros léxicos.
3. **`parser.js`** usa uma função para cada parte da gramática. A função `esperar()` devolve o token consumido; as funções de formas devolvem nós; `elementos()` devolve vetores; a função `analisarSintatico()` devolve a raiz `programa`.

4. **`main.js`** executa primeiro o léxico. Se não houver erros léxicos, tenta o sintático e apresenta a AST em JSON; caso contrário, mostra o primeiro erro sintático.
5. **`index.html`** oferece caixa de texto, botão de análise e duas saídas: tokens e resultado/erros.

Essa separação reproduz a progressão da aula: o parser deixa de devolver apenas sucesso ou falha e passa a construir valores que formam a árvore, de baixo para cima.

7. Casos de teste e resultados esperados

Arquivo	Esperado
`testes/validos/01_casa.txt`	AST válida: tela 480×320, quatro formas externas, um bloco com um retângulo e um bloco interno contendo uma reta.
`testes/validos/02_formas_modificadores.txt`	AST válida: tela 200×160, dois grupos com modificadores nas ordens inversas.
`testes/validos/03_atributos_repetidos.txt`	AST válida: último preenchimento `verde`, último traço `branco` e última espessura `2`.
`testes/validos/04_aninhamento.txt`	AST válida: três grupos aninhados, incluindo um bloco sem modificadores.
`testes/validos/05_zeros_e_hex.txt`	AST válida: aceita zeros, normaliza `#00ff88` para `#00FF88`.
`testes/invalidos/01_erro_lexico.txt`	Erro léxico na linha 2, trecho `#GG11AA`. Não há AST.
`testes/invalidos/02_falta_numero.txt`	Erro sintático na linha 2: esperado `NUM`, encontrado `TRACO`. Não há AST.
`testes/invalidos/03_grupo_vazio.txt`	Erro sintático na linha 3: esperado um elemento, encontrado `FECHA`. Não há AST.

Há ainda dez testes automatizados adicionais de casos-limite em `testes/executar-testes.mjs`. A execução local retornou **18 testes aprovados** (5 válidos + 3 inválidos + 10 adicionais). A página não tem dependências JavaScript externas e funciona com um servidor HTTP local, conforme o `README.md`.

8. Relação com os materiais da aula

O projeto respeita as quatro mudanças ensinadas em **Aula_AST_Robo.pdf**: `esperar()` devolve token; cada comando devolve nó; uma lista devolve vetor de nós; e a análise sintática devolve a raiz da AST. Também preserva a ordem das fases e todas as regras exigidas em **Atividade_Fase1_Linguagem_de_Desenho.pdf**, incluindo erros, gramática LL(1), dados numéricos e ausência de SVG na Fase 1.